Radar-Led, Camera-Assisted BEV-Language Model on nuScenes

A Complete Guide: Terminology, Foundations, Method, and Results

Project: Paper 1 - a radar-led, camera-assisted bird's-eye-view (BEV) language model that answers questions about a driving scene, built and evaluated on nuScenes. **Status as of:** 2026-10-22 (Part VII complete: the real-detector stress test, including four independent and isolated fix attempts for pedestrian-counting - slot balancing, detection threshold, confidence-score token, and detector retraining with a PR sweep - all four negative, pointing to a likely sensor-level limit for pedestrians rather than a fixable pipeline problem) **Code:** `D:\radar-bev-llm\code\radar\` (scripts) and `D:\radar-bev-llm\code\notebooks\` (01-06, annotated) **Repo:** https://github.com/amirbutt12/radar-bev-llm (branch `main`)

How to read this document

This is written so that someone with no background in self-driving sensors, radar physics, or language models can follow it start to finish. Part I explains every term before it gets used. Part II explains the existing research this project builds on. Part III walks through exactly what was built, using one running example scene the whole way through. Part IV tells the story of what was actually done, in order, including the mistakes and dead ends. Part V gives the results with every number explained in plain words. Part VI is the honest limitations and next steps as of the first version of this document. Part VII is a later addition: the real-detector stress test, which updates (and partly walks back) one of Part V's central claims. If a term ever feels unfamiliar later in the document, it was defined in Part I.

PART I - FOUNDATIONS: THE WORDS YOU NEED FIRST

Chapter 1: The driving scene and its sensors

A self-driving car (or a car being recorded to build a self-driving dataset) is covered in sensors so it can "understand" what's around it. This project uses two of them: camera and radar. A third, lidar, is explained here too because it matters to understand why it was deliberately left out.

Camera

A camera is exactly what it sounds like - it takes pictures. In self-driving data, cameras usually record video at around 12 frames per second. A camera's strength is that it captures rich detail: color, texture, text on signs, the exact shape of a pedestrian. Its weakness is that it is a 2D projection of a 3D world - a camera image alone does not directly tell you how far away something is or how fast it is moving. It also degrades badly in certain conditions: at night there isn't enough light, and glare, headlights, or rain on the lens can wash out or obscure what the camera sees.

Radar

Radar (RAdio Detection And Ranging) works completely differently from a camera. It sends out a radio wave and listens for the echo that bounces back off objects. From that echo it can work out:

- **Distance**: how long the echo took to come back (speed of light is constant, so time -> distance).
- **Velocity**: this is the part that matters most for this project, and it comes from the **Doppler effect**.

The Doppler effect, explained simply: you have heard this effect even if you didn't know its name. When an ambulance drives toward you with its siren on, the pitch sounds higher than normal; as it passes and drives away, the pitch drops. The sound wave gets "compressed" when the source moves toward you and "stretched" when it moves away. Radar

uses the exact same idea, but with radio waves instead of sound waves: if an object is moving toward the radar, the returning radio wave is compressed (its frequency shifts up); if it's moving away, the wave is stretched (frequency shifts down). By measuring that frequency shift, radar can tell you an object's velocity **directly, from a single snapshot in time** - no need to compare two separate images and work out how far something moved, the way a camera would have to.

This is radar's core physical advantage, and it is the entire reason this project exists: **radar measures speed directly; camera has to infer it indirectly by comparing multiple frames.** Radar is also largely unaffected by darkness (it doesn't need light, it sends its own signal) and works reasonably well in rain. Its weaknesses: the "picture" it returns is very sparse (a handful of reflection points, not millions of pixels) and it struggles to tell you what kind of object something is - a radar echo doesn't have a texture or a shape you can recognize the way a camera image does.

The specific radar used in this project is the kind actually fitted to the nuScenes recording cars: cheap, automotive-grade radar, not the dense, expensive "4D radar" used in some research datasets. This matters later when comparing this project to similar papers (Part II).

Lidar (mentioned, not used)

Lidar works like radar but with laser light instead of radio waves, and it does not usually measure velocity directly the way radar does (though some advanced lidar can). It produces a much denser point cloud, closer to a camera's level of detail, but it is also by far the most expensive sensor of the three, and - like camera - it can be degraded by weather (heavy rain or fog scatters the laser). **This project deliberately does not use lidar as a model input** - only radar and camera, matching the project's title, "radar-led, camera-assisted." Lidar is only used, indirectly, to look up where the car's camera was pointing at any given moment (a bookkeeping detail in nuScenes, not a sensor input to the model).

"Bird's-eye view" (BEV)

Imagine a drone hovering directly above the car, looking straight down. Instead of a normal photo (looking forward, with things in the distance appearing small and things nearby appearing large), you get a flat map: the car in the middle, and everything else - other cars, pedestrians, radar echoes - placed at their true position on that map, the same way a map of a city shows streets and buildings from directly above, not from eye level. This "top-down map" view is called **bird's-eye view, or BEV**. It's a natural representation for self-driving because it matches how you'd actually want to reason about the scene ("that car is 10 meters ahead and 2 meters to the left"), and because it's the natural shape radar data already comes in.

Chapter 2: nuScenes, the dataset

What a dataset is, and why you need one

To teach a computer to understand driving scenes, you need many real examples of driving scenes, each one labeled with the "ground truth" (the correct answer) - where the cars and pedestrians actually were, how fast they were actually moving. A **dataset** is this collection of recorded, labeled examples. **nuScenes** is a widely used public self-driving dataset, recorded in Boston and Singapore, with camera, radar, and lidar all recording at once, plus human-drawn boxes around every car, pedestrian, cyclist, etc.

Scenes, keyframes, and sweeps

nuScenes is organized like this:

- A **scene** is one continuous recording, about 20 seconds long, of the car driving somewhere.
- Within a scene, every sensor keeps recording constantly - camera at about 12 frames/second, radar at about 13 scans/second. Each individual recorded moment is called a **sweep**.
- But drawing boxes around every object by hand, for every single sweep, would take forever. So humans only labeled the scene at 2 moments per second. Each of those labeled moments is called a **keyframe**. A keyframe is a "real" sample with full ground-truth annotations; the sweeps in between keyframes have raw sensor data but no hand-drawn boxes.

Think of it like a flipbook: the whole flipbook is the scene, every single page is a sweep, and every 6th or 7th page (about every half-second) has been carefully labeled by a person - that's a keyframe.

Full dataset size, and the "mini" vs "trainval" split

The full nuScenes dataset has **1,000 scenes total**: 850 are released with full annotations for building and testing models (called **trainval**), and 150 are held back for an official competition leaderboard (called **test**, not used in this project). Of the 850 trainval scenes, nuScenes itself designates **700 as the official training set and 150 as the official validation set** - this is nuScenes' own fixed split, not something this project invented. In total, the 850 trainval scenes contain **34,149 keyframes**.

Because downloading and processing all 850 scenes takes a long time, nuScenes also ships a much smaller **"mini"** version: just **10 scenes, 404 keyframes**, meant only for quickly testing that your code runs correctly before scaling up to the real thing. A large part of the story in this document (Part IV) is about starting on mini, discovering that 10 scenes genuinely isn't enough data to learn from, and then scaling up to the full 850-scene trainval set.

Scene conditions: day, night, rain

Each scene has a short text description written by nuScenes' creators, for example "Night, big street, bus stop, high speed" or "Night, after rain, many peds." This project uses that text to tag each scene as **night**, **rain**, or **clear**, which becomes important later: the whole point of "radar-led" is that radar should help most exactly in the conditions where camera struggles, so being able to split results by condition is essential.

Annotations: boxes and categories

For every labeled object in a keyframe, nuScenes provides a **3D bounding box** (its position, size, and the direction it's facing) and a **category** (what kind of thing it is - car, pedestrian, bicycle, etc.). This project only uses two categories: **car** and **pedestrian**.

Chapter 3: Turning radar points into something a computer can learn from

Point clouds

Radar does not return an image. It returns a list of individual reflection points. Each point is a small bundle of numbers. In this project, each radar point has 8 numbers: its position (x, y, z - how far forward, how far left/right, how far up), its velocity (vx, vy - speed in each direction, already corrected for the car's own motion, and v_r - the raw radial velocity straight from the Doppler measurement), its radar cross-section (rcs - roughly, how strongly it reflected the radar signal, related to how big/metallic the object is), and dt (how long ago, in time, this particular point was recorded, since points come from several recent sweeps - see below). A single keyframe typically has only a few hundred to a couple thousand of these points total, covering the entire area around the car - genuinely sparse compared to a camera's millions of pixels.

Multi-sweep accumulation

A single radar sweep at one instant is extremely sparse - often only a handful of points land on any given object. The fix used here (a standard nuScenes technique) is to take not just the keyframe's own sweep, but the **5 most recent sweeps before it** (going back roughly half a second) from **all 5 radars** mounted on the car (front, front-left, front-right, back-left, back-right), and combine them all into one bigger point cloud for that keyframe. Think of it like taking 5 photos in quick succession of a lightly falling snow and overlaying all 5 - any single photo barely shows the snow, but stacked together, a clear pattern emerges. Because the car itself moved slightly between those sweeps, each point first has to be mathematically shifted into a single shared reference frame (explained next) before they can be combined.

Coordinate frames ("ego frame")

Every sensor reading starts out described relative to wherever that specific sensor is mounted on the car. To combine radar points from 5 different radars, recorded at 5 slightly different moments, they all need to be converted into one

shared coordinate system. The natural choice is the **ego frame**: treat the car itself, at the exact moment of the keyframe, as the center of the universe, with "forward" and "left" as the two main directions. Every point - no matter which radar it came from or how many fractions of a second ago it was recorded - gets mathematically translated and rotated into "where was this, relative to the car, right now." This is a straightforward (if fiddly) application of the car's own recorded position and orientation (nuScenes calls this an **ego pose**) at each moment.

Pillars

Once all the radar points for a keyframe are sitting in one shared ego-frame point cloud, they need to be turned into a shape a neural network can process efficiently. The chosen approach is to lay a grid over the bird's-eye view, like dividing a parking lot into a checkerboard of small squares. In this project the grid is **128 x 128 cells**, each cell **0.8 meters** across, covering a **102.4 meter** window around the car (51.2 m to each side). Each one of these 3D vertical columns of space - one grid cell, extending from the ground up - is called a **pillar**. Every radar point gets sorted into whichever pillar its (x, y) position falls into.

Two ways of summarizing what's inside each pillar were used in this project:

- **Hand-crafted pillars** (used early on, mainly for visualizing and sanity-checking): for each pillar, compute 7 simple numbers directly - how many points landed in it, the average vx/vy/v_r/rcs of those points, the biggest rcs seen, and the average dt.
- **Learned pillar encoder** (the one actually used in the final model): instead of hand-picking those 7 summary numbers, a small neural network (described in the next chapter) looks at all the raw points in a pillar and learns, from data, what summary is actually useful - producing 64 numbers per pillar instead of 7. This "let the network figure out what matters" approach generally works better than hand-picked summaries, at the cost of needing to be trained.

Either way, the result is a **128 x 128 grid of little feature vectors** - something that looks structurally just like an image (a grid of cells, each with some numbers), which is exactly the shape of input that the next kind of neural network, described below, is built to process.

Chapter 4: Neural networks, briefly

A full explanation of neural networks is outside the scope of this document, but a few terms are used constantly and need a plain-word definition.

- **Neural network**: a function with many adjustable internal numbers (**parameters** or **weights**) that learns, from many examples, to turn an input (like the 128x128 radar grid) into a useful output (like "there's a car centered here"). "Training" means repeatedly showing it examples and nudging its parameters so its outputs get closer to being correct.
- **CNN (convolutional neural network)**: a kind of neural network specifically designed for grid-shaped data (like images, or the radar pillar grid), which looks at small local neighborhoods of cells at a time and slides that same small "lens" across the whole grid.
- **U-Net**: a specific, very common CNN shape, used as this project's detection backbone. The idea: first **shrink** the grid down repeatedly (compressing 128x128 down to smaller and smaller grids), which forces the network to summarize "what kind of stuff is roughly around here" rather than getting lost in fine detail - this is like zooming a map out from street level to city level to find roughly where something is. Then, the network **grows the grid back up** to its original size, step by step, while also referring back to the detailed, un-shrunk version from the first half - like zooming back in from the city level to the exact street, now that you know roughly where to look. The result is a grid the same size as the input, but where each cell's numbers now encode a rich understanding of both the local detail and the broader context around it.
- **Backbone**: a general term for "the main neural network that turns raw-ish input into a useful internal representation," before any task-specific head is applied. In this project, the U-Net is the detection backbone.

Chapter 5: Object detection

What detection means here

Object detection means: given the processed radar grid, find every car and pedestrian, and report where each one is, how big it is, and which way it's facing.

CenterNet-style targets

Rather than directly predicting box corners (an older, more complicated approach), this project uses the **CenterNet** idea: train the network to predict a **heatmap** - like a weather map showing hot spots - with one "hot spot" exactly at the center of each object. A perfect car at a certain grid cell should produce a bright spot right there, fading out smoothly in the cells around it (a small blurred blob, not a single hard pixel, which makes training easier). Separately, at that same center cell, the network also predicts the object's exact size, precise position within the cell, and facing direction (as a sine/cosine pair, a standard trick for representing an angle smoothly). At evaluation time, the heatmap is scanned for local bright spots, and each one is read off as one detected object.

Average Precision (AP)

To grade how good a detector is with a single number, this project uses **Average Precision at 2 meters**: a predicted object counts as "correct" if it's within 2 meters of a real, labeled object of the same class (car or pedestrian). AP then summarizes, across all the model's predictions at every confidence level, how good the trade-off is between catching real objects and avoiding false alarms. An AP of 1.0 would mean a perfect detector; an AP of 0.0 would mean the detector is essentially useless. As a rough feel: in this project, the detector's AP went from essentially memorizing a tiny training set (very high training AP, very low validation AP - a classic sign the model hasn't actually learned anything generalizable, just memorized the few examples it saw) up to a real, meaningful AP once enough data was used (see Part V).

Mirror augmentation

A simple, free way to effectively give the model more training examples: take every training radar grid and its matching ground-truth boxes, and flip them left-right (mathematically, this means negating the y-position, the sideways velocity, and the sideways component of the facing angle). A street scene flipped left-right is still a perfectly valid, physically correct street scene, so this "costs" nothing and was confirmed to help (see Part V).

Chapter 6: Language models and how they answer questions

Tokens and large language models (LLMs)

A **large language model (LLM)** is a neural network trained on enormous amounts of text to predict, one small chunk at a time, what word or word-fragment should come next. Each one of those small chunks is called a **token** - sometimes a whole word, sometimes just a piece of one. An LLM "writes" an answer by repeatedly predicting the single most likely next token, adding it to what's been written so far, and repeating.

This project uses a relatively small, publicly available LLM called **Qwen2.5-0.5B-Instruct** ("0.5B" meaning roughly half a billion internal parameters - small by modern standards, chosen deliberately for fast, cheap experimentation rather than peak quality).

Fine-tuning and LoRA

An LLM straight off the shelf doesn't know anything about radar grids or this project's specific question format. **Fine-tuning** means further training an already-trained model on new, task-specific examples so it adapts. Fully fine-tuning every one of an LLM's half a billion numbers is expensive and can also make the model forget other things it used to know. **LoRA** (Low-Rank Adaptation) is a cheaper alternative: instead of touching the model's original half a billion numbers at all, you freeze them completely and add a small number of brand-new, extra numbers (in this project, about 2.16 million - under 0.5% the size of the original model) alongside the existing ones, and only train those new ones. A simple way to picture it: instead of rewriting an entire textbook to correct it, you leave the textbook exactly as printed, and attach small sticky notes with corrections next to the relevant pages - cheaper, faster, and you can always peel the sticky

notes off and get the original textbook back.

Feeding radar and camera into a language model: projectors

An LLM, by itself, only understands sequences of text tokens - it has no built-in way to read a 128x128 radar grid or a camera image. The fix used throughout this project is to turn each non-text input into something that *looks* like a sequence of tokens to the LLM, through a small extra neural network called a **projector**. For radar: the 128x128 grid of 64-number pillar features is shrunk down to a 16x16 grid (256 "patches," each covering a 6.4m x 6.4m chunk of the map) and each patch's 64 numbers are passed through a small projector network that reshapes them into the same size numbers the LLM's own word-tokens use. The result is 256 "radar tokens" that get placed in front of the question, as if they were extra words the model can "read" before answering. Camera images and the explicit per-object motion tokens (next section) use their own separate, similarly-shaped projectors. Each projector also has a small learned "which patch is this" position signal attached to each token, the same way a page number helps you know where in a book you are.

Explicit object (motion) tokens

Squeezing the whole scene down into a 16x16 grid of radar tokens, as above, can blur away small or sparse details - a lone pedestrian can easily get lost when the map is compressed that much. The fix explored in this project: instead of relying only on the blurry whole-scene grid, also give the model a small number of dedicated tokens, one for each of up to 8 nearby objects, each one carrying that specific object's class (car or pedestrian), its position, and - critically - **the actual measured average velocity of the real radar points that landed inside that object's box** (not a label, an honest measurement, possibly imperfect if few or no radar points happened to land on that object). This is explained in full technical detail in Part III, with a worked example.

Chapter 7: How we test whether a model is really "looking," not just guessing

This is maybe the single most important methodological idea in this whole project, so it gets its own chapter. A language model can produce an answer that *sounds* plausible without actually having looked at - or used - the sensor data it was given. For example, if 80% of "are there any cars?" questions in the training data have the answer "yes," the model might just learn to always answer "yes" regardless of what the radar actually shows, and still score well. Four separate checks were used throughout this project to guard against mistaking "the model sounds right" for "the model is actually using the sensor":

- **Majority-answer baseline**: before even looking at the trained model, compute what accuracy you'd get by simply always answering every question with whatever the single most common correct answer was for that question type, in the training data. No model, no sensors - pure statistics. Every real result in this project has to clear this bar to mean anything at all.
- **Wrong-frame control**: after training a model normally, test it again, but secretly swap in the radar/camera data from a **different, random moment** instead of the correct one, while keeping the question the same. If the model's accuracy barely drops when given the wrong sensor data, it means the model wasn't really using that sensor's specific content for the right answer - it was relying on something else, like typical patterns in the question wording. If accuracy drops clearly, that's good evidence the model really was reading the correct frame's content. The gap between "correct frame" accuracy and "wrong frame" accuracy is the single most important number reported throughout this project.
- **Zeroing ablation**: a more surgical version of the above - instead of swapping in a different real frame, just blank one modality's tokens to all zeros (as if that sensor wasn't there at all), while leaving the others correct. This isolates exactly which modality a multi-sensor model depends on most. In practice, this check turned out to be unreliable in this project (explained honestly in Part IV and Part VI) - likely because a model never sees all-zero input during training, so "all zeros" is a strange, unfamiliar input that can confuse the model in ways unrelated to how much it actually values that sensor, the same way you'd likely behave strangely if a sense suddenly, inexplicably went numb (as opposed to a controlled, fair test).
- **Day/night/rain condition split**: compute every check above separately for scenes tagged night, rain, or clear, to test the actual core hypothesis of this whole project - that radar's advantage should show up most specifically in the conditions where camera struggles.

PART II - WHERE THIS FITS IN EXISTING RESEARCH

The specific question this project asks

Stated precisely, now that Part I has defined every term in it: **can radar meaningfully improve a language model's ability to answer questions about a driving scene, specifically in the conditions (night, rain) where radar's physical advantage over camera - direct velocity measurement, independence from visible light - should matter most?**

This is deliberately narrower than "is radar useful for self-driving" in general, because that broader claim is already well established for one specific task: object detection.

Radar helps detection - already well established

Several existing papers combine radar and camera specifically to improve object **detection** (finding and boxing objects, not answering questions about them):

- **CenterFusion** (Nabati & Qi, 2021) associates radar detections with objects found in a camera image, and uses that radar information to improve the camera-only detector's estimates of depth and velocity - it reported roughly a 12% improvement over a camera-only baseline on nuScenes.
- **RCBEVDet** (2024) is a more recent radar-camera fusion approach, also for detection, also evaluated on nuScenes.

These confirm radar+camera fusion is worth doing for detection. This project's job is to find out if the same kind of benefit shows up for *language* - for a model answering questions, not drawing boxes.

Existing driving language benchmarks are camera/LiDAR-centric

- **NuScenes-QA** is the question-answering benchmark this project's evaluation protocol is directly built on (same style of exact-match accuracy, same style of question categories like exist/count/object/status/ comparison). Its own published baselines combine camera or LiDAR detectors with a separate question-answering module; radar is not part of its baselines at all.
- **DriveLM** is a broader, more recent driving question-answering benchmark (graph-structured, covering perception/prediction/planning questions), also built entirely on camera and LiDAR, with no radar.

Both of these represent "the mainstream approach" a skeptical reader might reasonably ask about: why not just use camera and LiDAR, like everyone else? This project's answer is specifically the day/night evaluation in Part V - these benchmarks never test whether their camera/LiDAR-centric approach actually degrades in the conditions where radar should help.

The closest sibling work - and how this project differs

Two papers, both from mid-to-late 2026, attempt something close to this project's idea, and deserve an honest, specific comparison rather than a vague "we're different":

- **Radar4D-VLM** is a radar-only language model (no camera at all) that reads radar and answers questions about a scene. It differs from this project in two important ways: it uses dense, expensive **4D radar** sweeps (far richer, more detailed radar data than nuScenes' sensors provide), and it has no camera-fusion component at all - so it cannot test the "radar-led, camera-*assisted*" framing this project is specifically built around.
- **RLM** (Radar Language Model) is trained on 800,000+ radar-caption pairs, but entirely from the **CARLA driving simulator**, not real recorded data, and it produces free-form captions rather than being scored with the kind of exact-match, ground-truth-checked accuracy this project uses.

This project's specific, deliberate combination - real nuScenes data (not simulated), cheap sparse automotive radar (not dense 4D radar), radar *and* camera together (not radar-only), and a rigorous attribution methodology (Part I, Chapter 7) specifically designed so "the model sounds plausible" cannot be mistaken for "the model is actually using the sensor" - is not directly duplicated by either of these.

Scope decisions made along the way

Two scope questions came up early and were deliberately settled rather than left open: whether to add lidar as a third sensor (decided: no, stay radar+camera, matching the project's title and avoiding months of extra engineering for a sensor the physical-advantage story doesn't need), and whether to move to a dataset with more severe weather like fog or snow, such as K-Radar (decided: no, stay on nuScenes, since nuScenes only tags night and rain, not fog/snow, but switching datasets would mean abandoning the entire built pipeline for a much rougher, less mature one).

PART III - WHAT WE BUILT, STEP BY STEP, WITH A WORKED EXAMPLE

This chapter walks through the entire pipeline exactly as built, following one concrete example the whole way through: a single keyframe from scene `scene-0061`, a few seconds into the recording, with one car driving ahead and one pedestrian crossing nearby.

Step 1 - raw radar sweeps. For this keyframe, the system gathers the 5 most recent sweeps from each of the 5 radars - 25 sweeps in total - each one a small list of echo points, each point still described relative to whichever radar recorded it, at whatever fraction of a second it was recorded.

Step 2 - accumulate into the ego frame. Every one of those points gets mathematically shifted into "where was this, relative to our car, at the exact moment of this keyframe" (Chapter 3). The 25 sweeps collapse into one combined point cloud for this keyframe - in a typical keyframe, somewhere between a few hundred and about two thousand points.

Step 3 - pillarize. That combined point cloud gets sorted into the 128x128 grid of 0.8m-wide ground cells (Chapter 3). The pedestrian, being small, might end up with only 2-3 radar points landing in her pillar; the car ahead might have a dozen or more.

Step 4 - learned encoding. A small neural network (the pillar encoder, Chapter 3-4) looks at the points in each occupied pillar and produces 64 summary numbers per pillar, turning the grid into a 128x128x64 "pseudo-image."

Step 5 - detection backbone and (optionally) detection. The U-Net backbone (Chapter 4) processes that pseudo-image into a richer internal representation. A detection head reads that representation and finds the car and pedestrian as two heatmap "hot spots" (Chapter 5) - this part of the pipeline is also trained and evaluated on its own, as a sanity check that the radar features are meaningful before anything language-related touches them (see Part V, detection results).

Step 6 - cache the dense radar features. Rather than re-running the (expensive) pillar encoder and backbone every single time, the 64x128x128 feature map from just before the detection heads is average-pooled down to 64x32x32 and saved to disk once per keyframe. This cached feature map is what the language stage actually reads.

Step 7 - explicit object (motion) tokens. Separately, for this same keyframe, the system looks at the (up to 8) nearest known car/pedestrian boxes, nearest first. For the car, it finds which of the keyframe's raw radar points fall inside (or very near) its box, and averages their measured vx/vy - producing a genuine, radar-measured velocity estimate for that specific car. For the pedestrian, suppose only 1 radar point happened to land near her - the system still computes an average velocity from just that one point, and also honestly records `has_radar_points = 1` (or, if she'd had zero nearby points, the token would honestly report `has_radar_points = 0` and a velocity of exactly zero, rather than making something up). Each object becomes one small 8-number token: `[is_car, is_pedestrian, x, y, vx, vy, speed, has_radar_points]`.

Step 8 - camera tokens. The front camera's image for this same keyframe is passed through a frozen, pre-trained vision network called **CLIP**, which was never trained on anything to do with this project - it's a general-purpose, off-the-shelf image understanding network, used here exactly as-is. It cuts the image into a 7x7 grid of patches and describes each patch with 768 numbers, giving 49 "camera tokens," also cached to disk once per keyframe.

Step 9 - assembling a question. Suppose the question for this keyframe is "How many pedestrians are moving in the scene?" The system places, in order: the 256 radar tokens (if this model uses radar), the 49 camera tokens (if this model uses camera), the up to 8 object tokens (if this model uses them), and then the question itself, written out as actual words ("Question: How many pedestrians are moving in the scene?\nAnswer:"), each word broken into the LLM's own tokens. Every one of the radar/camera/object tokens has already been reshaped, by its own small projector network (Chapter 6), into the same "shape" as a normal word-token, so from the LLM's point of view, it's just reading one long sequence of tokens - some representing sensor content, some representing the actual written question - and it generates its answer, one token at a time, the normal way any language model does.

Step 10 - scoring. The model's generated answer ("1", say) is compared, as plain text, to the true answer recorded in the dataset. An exact text match counts as correct; anything else counts as wrong. No partial credit.

Any of radar, camera, or object tokens can be switched on or off independently for a given model - this is exactly what makes the ablation experiments in Part IV and Part V possible: train several versions of "the same" model, each with a different subset of the three token types present, and compare.

PART IV - THE JOURNEY: WHAT WE ACTUALLY DID, IN ORDER

This section tells the story in the order it happened, including the dead ends, because the dead ends are part of the evidence for the final conclusions.

Phase A - building the pipeline on mini, and immediately finding its limit

Everything in Part III, Steps 1-6, was built and tested first on nuScenes' small 10-scene "mini" dataset (Chapter 2), including mirror augmentation (Chapter 5). The detector's accuracy on the small set it was trained on was nearly perfect, while its accuracy on held-out validation data was very poor - the classic signature of a model that has simply memorized its tiny training set rather than learned anything that generalizes. This was expected, not a bug: 8 training scenes is not enough data for a neural network of this size, and it immediately motivated eventually scaling up (Phase E).

Phase B - first attempt at radar-to-language, on mini

The full Steps 1, 2, 3, 4, 6, 9, 10 pipeline (radar only, no camera or object tokens yet) was built and tested on mini, using the existing NuScenes-QA question set (Part II). The wrong-frame control (Chapter 7) showed almost no gap - an early warning sign that the model wasn't clearly reading the radar's actual content yet, though still on far too little data to be sure.

Phase C - adding camera, and the first clean negative result

Camera tokens (Part III, Step 8) were added, and the full blind / radar-only / camera-only / fusion comparison was run. Fusion (radar+camera together) showed the first real wrong-frame gap seen in the whole project - small, but real. Then, to directly test radar's actual physical selling point (Chapter 1's Doppler explanation), three brand new question types were invented and auto-generated straight from the dataset's own ground-truth object velocities (not from NuScenes-QA, which never asks about speed, only "moving or parked"): "how many cars are moving," "how many pedestrians are moving," and "is the fastest car slow or fast." **The result was a clean, honest negative**: every single model, radar included, scored at or below the "just guess the most common answer" baseline on all three new questions. The diagnosis: squeezing the whole 128x128 radar grid down to a 16x16 token grid (Part III, Step 6-9) blurs away exactly the fine, per-object detail needed to count individual moving things.

Phase D - explicit object tokens, as the fix for Phase C

Part III's Step 7 (explicit, per-object, radar-measured motion tokens) was built specifically to address Phase C's finding. On mini, it produced a genuine but modest, mixed result: the best score yet on car-counting, but still worse than guessing

on the speed question when tested alone, without the dense map alongside it. The honest read at the time was that this looked like another data-scale limit (only a few hundred training examples of each new question type, from just 6 scenes), not a wrong idea - motivating Phase E.

Phase E - scaling up to the full 850-scene trainval dataset

The full nuScenes trainval dataset (Chapter 2) was downloaded. One genuine surprise along the way: nuScenes ships its data in 10 download "parts," and **night/rain scenes turned out not to be evenly spread across those parts** - an early partial download (the first 3-5 parts, 263-433 scenes) had almost no night coverage at all, repeating the exact same problem mini had. Only after downloading all 10 parts (getting all 850 scenes) did the dataset finally have a workable day/night split: 700 official training scenes (84 of them night, 138 rain) and 150 official validation scenes (15 night, 27 rain) - finally enough to actually test the project's core hypothesis.

The radar point cache (Part III, Steps 1-3) was rebuilt for all 34,149 keyframes (a slow, roughly 11-hour, single-threaded process), the camera cache likewise, and the detector (Part III, Step 5) was retrained on the full 700-scene training set. Its validation accuracy came out far higher than mini's memorization-only result (see Part V) - confirming the detector itself had never been the real bottleneck, just the amount of data it had to learn from.

Phase F - the baseline ladder at full scale: still flat

The blind / radar-only / camera-only / fusion comparison (Phase C's setup) was repeated at full scale, on the *original* NuScenes-QA questions (not yet the new motion questions). The result was still disappointingly flat: fusion barely beat blind, and radar-only showed almost no wrong-frame gap. Training for more epochs (passes through the data) showed accuracy climbing and then leveling off, ruling out "it just needs more training time" as the simple explanation.

Phase G - the motion-question ablation at full scale: the headline result

The three new motion/speed question types from Phase C were regenerated at full scale (84,318 training questions, 18,033 validation questions - comfortably enough data this time) and a full six-way comparison was trained: blind, camera-only, dense-radar-only, object-tokens-only, radar+object-tokens (no camera), and full fusion (radar+camera+object tokens) - all six on the exact same randomly chosen 60,000 training questions and 6,000 validation questions, so every number is directly, fairly comparable. **This produced the first unambiguous evidence in the entire project that the model genuinely uses radar content, specifically concentrated on pedestrian motion counting** - the full numbers are in Part V.

Phase H - the day/night/rain test: the actual core hypothesis

With the full-scale night/rain data finally available (Phase E) and a model finally showing a real signal to test (Phase G), the wrong-frame control was recomputed separately for clear, night, and rain scenes, for every model in the Phase G comparison. The result (Part V) is the clearest support the project has found yet for its original starting hypothesis.

Infrastructure problems, found and fixed along the way

A few real engineering problems came up during this process, worth recording honestly because they affect how much confidence to place in certain numbers:

- **A genuine data bug**: partway through, six already-trained models were accidentally being re-evaluated on the wrong (smaller, easier) set of questions, because of a small mismatch between an old saved results file and a newly added setting. This was caught by comparing against the original training logs *before* any of the affected numbers were used anywhere - nothing wrong ever made it into a reported result.
- **A memory problem**: the very first version of the data-loading code tried to hold every keyframe's radar and camera features in memory at once, which needed roughly 7 gigabytes at full scale - enough, on this machine's 16 gigabytes of RAM, to make a training run stall for many hours without ever actually crashing, just sitting idle. Fixed by reading each keyframe's features from disk only when actually needed, instead of loading everything upfront.
- **GPU driver instability**: twice during this project, the graphics card driver crashed or hung mid-training. One root cause was an outdated driver (fixed by updating it); the other, more interesting cause was running two GPUs at once under heavy training load at the same time, which once caused the entire computer to crash and reboot. After that

was understood, every subsequent run used one GPU at a time, and nothing has crashed since. As a safety net against this kind of event, the training code was also changed to save its progress after every single pass through the data, instead of only at the very end, so a future crash would lose minutes of work, not hours.
- **The zeroing ablation (Chapter 7) turned out to be unreliable**: testing "which sensor matters most" by blanking one sensor's tokens to all zeros gave inconsistent answers across otherwise-very-similar runs - in one run, zeroing the radar tokens was catastrophic and zeroing camera barely mattered; in other runs, the opposite. The likely explanation (Chapter 7) is that an all-zero input is simply unlike anything the model ever saw during training, so the test partly measures "how confused does the model get by a bizarre input" rather than cleanly measuring "how much does the model rely on this sensor's real content." The wrong-frame control stayed the trustworthy signal throughout, and is what every headline number in Part V is based on.

PART V - RESULTS, EXPLAINED

5.1 Detection (the radar backbone)

	mini (8 training scenes)	**full trainval (700 training scenes)**
car AP@2m	train ~0.97 / validation 0.07-0.12 (memorized, didn't generalize)	validation **0.385**
pedestrian AP@2m	validation ~0.01	validation **0.120**

In plain words: on mini, the detector essentially memorized its 8 training scenes and had almost no ability to find cars or pedestrians in scenes it hadn't seen before. With the full 700 training scenes, it found real cars roughly 3x as often (and a meaningfully higher fraction of pedestrians too, though pedestrians remain the harder of the two classes throughout this whole project, being smaller and sparser in radar). This confirms the detector itself was never the thing holding back the language results - there was always real, useful radar signal available once there was enough data to learn it from.

5.2 Mini-scale language results

model	**overall accuracy**	**wrong-frame gap**
majority-guess baseline	0.407	n/a
blind (no sensors, question text only)	0.502	n/a
radar-only	0.531	+0.011
camera-only	0.505 (best single epoch: 0.530)	+0.004
radar+camera fusion	0.531 (best single epoch: 0.551)	**+0.019**

Every model beat blind-guessing by a meaningful amount - but the wrong-frame gaps (the real test of "is it actually using the sensor") were all tiny, fusion's being the largest found anywhere on mini. On the new motion/speed questions specifically, every single model, radar included, scored at or below the plain majority-guess baseline - a clean negative result (Phase C).

5.3 Full trainval baseline ladder (original NuScenes-QA questions only)

model	**final accuracy**	**wrong-frame gap**
majority-guess baseline	0.365	n/a
blind	0.554	n/a
radar-only	0.523	+0.002
fusion (radar+camera+object tokens)	0.562	+0.029

Still a fairly flat picture - more data alone, on the *original* question set, did not produce a strong, clear signal that the model leans on radar.

5.4 Full trainval motion-question comparison - the headline result

All six models below were trained on the exact same randomly chosen 60,000 training questions and evaluated on the exact same 6,000 validation questions, so every number is a fair, direct comparison. The "majority-guess" baselines for these three new question types are: how-many-cars-moving 0.195, how-many-pedestrians-moving 0.34, fastest-car-slow-or-fast 0.345.

model	overall accuracy	wrong-frame gap	cars-moving	peds-moving	speed
blind (no sensors)	0.513	n/a	0.19	0.34	0.35 (matches guessing almost exactly, as it should)
camera alone	0.544	+0.053	0.39	**0.33**	0.68 (fails only on counting pedestrians)
radar (dense map) alone	0.545	+0.041	0.35	0.36	0.66 (its weakest point is also pedestrians)
object tokens alone	0.520	+0.055	0.29	0.43	0.60 (best of the three "alone" models on pedestrians, worst on cars)
radar + object tokens	**0.558**	**+0.075**	0.37	0.45	**0.73** (best or tied-best on every single number)
full fusion (+camera too)	0.552	+0.070	0.39	**0.47**	0.68 (best of all on pedestrians specifically)

What this means, in plain words:

First, the blind model landing almost exactly on the plain-guessing baseline for all three question types is an important, reassuring sign: it confirms these specific questions genuinely cannot be answered just from clever guessing about typical phrasing - a model that scores above that baseline on these particular questions really must be reading something real.

Second, and this is the project's central, specific finding: **camera alone is the only model that fails to beat plain guessing on counting moving pedestrians** (0.33 against a 0.34 baseline - essentially no better than a coin flip's worth of knowledge). Every single model that includes radar in any form clears that same bar clearly (0.36 to 0.47). On the other two question types (counting moving cars, and judging whether the fastest car is slow or fast), camera actually keeps pace with radar reasonably well. So the fair, precise claim is not the vague "radar helps" - it's the specific, testable claim that **radar's distinctive value shows up in detecting that pedestrians are moving**, which makes physical sense: pedestrians are small and easy for a sparse radar image to lose track of visually-speaking, but a radar echo's Doppler shift tells you directly whether something is moving, regardless of how small or hard-to-see it is.

Third, a smaller but genuinely interesting finding: the dense radar map and the explicit per-object tokens are **complementary, not two versions of the same thing**. The dense map's single weakest spot is exactly pedestrian counting (where it barely beats guessing, 0.36 vs 0.34); the object tokens' weakest spot is cars and speed. Used together (with no camera at all), they form the best single model on every metric measured. Adding camera on top of that combination doesn't improve things further - its only edge is a slightly better pedestrian score, well within the kind of small, run-to-run variation you'd expect by chance at this sample size.

5.5 Day, night, and rain - the actual test of the original hypothesis

Same validation set as above (6,000 questions), now split by each keyframe's scene condition: 4,660 clear, 353 night, 987 rain.

model	wrong-frame gap, clear	wrong-frame gap, night	wrong-frame gap, rain
full fusion (+camera)	+0.064	**+0.144**	+0.068
radar + object tokens	+0.069	**+0.139**	+0.078
object tokens alone	+0.050	+0.110	+0.053
radar (dense map) alone	+0.036	+0.079	+0.054
camera alone	+0.045	+0.091	+0.079

In plain words: for every single one of these five models, the wrong-frame gap (how much the model's accuracy drops when it's given the wrong frame's sensor data instead of the right one) is clearly bigger at night than in clear conditions - roughly double, in most cases. That means every model leans harder on getting the correct sensor reading specifically at night, when conditions are harder. And the two models that include radar together with object tokens have the two largest night gaps of all, bigger than camera alone's. This is honestly the strongest piece of evidence the whole project has found for its original starting idea - that radar's value is concentrated exactly where camera should be expected to struggle.

One caveat has to be stated clearly: night only makes up 353 of the 6,000 validation questions (about 6%), so the *exact size* of these numbers carries real statistical uncertainty - with a sample this size, any single number could plausibly be off by roughly 0.03 in either direction just from which specific questions happened to be randomly chosen for validation. So the precise numbers above should be read as "roughly this big," not as exact measurements. What's more convincing is the *consistency of direction*: all five separate models, trained completely independently of each other, show the same pattern (night bigger than clear) - if this were purely random noise rather than a real effect, you would not expect all five to agree on the direction by chance.

PART VI - HONEST LIMITATIONS AND WHAT'S NEXT

Limitations, stated plainly

- **The object tokens use an "oracle" shortcut.** Right now, the explicit per-object motion tokens (Part III, Step 7) are placed using the dataset's own ground-truth object boxes - a real, deployed system wouldn't have those available at the moment it needs to answer a question; it would only have its own trained detector's predictions, which are noisier and sometimes wrong or missing. This is the single most important thing left to fix: the pedestrian-counting finding in Part V needs to be re-checked using the project's own detector's predictions in place of ground truth, to see whether the finding survives contact with a realistic pipeline.
- **The training set used was a subsample, for practical reasons.** The motion-question models in Part V, Section 5.4-5.5 were trained on 60,000 of several hundred thousand available questions, for 2 passes through the data - a deliberate choice to keep each experiment's running time manageable, not a claim that this is the best possible amount of training. Using more data, or more passes, is the most direct way to sharpen these numbers further and shrink the night-condition uncertainty described above.
- **nuScenes only tags night and rain, not fog or snow.** Any claim in a future paper about "adverse weather" should be scoped precisely to night and rain, since that is genuinely all this dataset can test.
- **Most trainval-scale comparisons are single training runs**, not repeated and averaged across multiple random seeds. Earlier, smaller-scale experiments on mini (the mirror-augmentation result) were repeated across 3 seeds and showed roughly 0.01-0.02 of natural run-to-run variation; this has not been separately re-measured at full trainval scale.
- **The zeroing ablation (Chapter 7) should not be trusted as primary evidence**, for the reasons explained in Phase H's infrastructure notes - it gave inconsistent results across similar runs. The wrong-frame control is the methodology this project's conclusions actually rest on.

Recommended next steps, in priority order

Note (2026-10-21): item 1 below has since been attempted - see Part VII. It replaced the oracle boxes with the detector's real predictions and found the pedestrian-counting result does not survive as-is, and that two cheap fixes (class-balanced slots, a lower detection threshold) do not recover it either. The list below is kept for the record; Part VII's "Updated priorities" section is now the current priority order.

1. Replace the oracle ground-truth object boxes with the project's own trained detector's real predictions, and re-run the Part V, Section 5.4-5.5 comparison to see whether the pedestrian-counting and night-gap findings survive. **[Done, 2026-10-21 - see Part VII. Result: car-counting and speed judgment survive; pedestrian- counting does not, and remains unresolved.]**
2. Scale up the training data and number of passes used for the motion-question models, both to sharpen the headline numbers and to shrink the statistical uncertainty around the night-condition result.
3. Repeat the key comparisons (at minimum: radar+object-tokens vs. camera-alone vs. blind, on the motion questions, in clear vs. night conditions) across multiple random seeds, and report a confidence interval or significance test on the headline gaps, rather than single numbers.
4. Write the pedestrian-counting finding and the night-gap-doubling finding into the paper's results chapter as the two central, specific, falsifiable claims this project can now defend - a clear step up from the flat "radar might matter somewhere" picture that mini's small scale could only gesture at, never actually show. **[Revised by Part VII: the pedestrian-counting claim can only be stated under oracle conditions for now, not as a general result - see Part VII's closing section.]**
5. If pursuing larger, multi-GPU training runs again in the future: either stay with one GPU at a time (the approach that has been reliably crash-free), or specifically investigate and resolve the two-GPU crash cause first before relying on parallel training again.
6. Consider whether a larger language model backbone (something bigger than the current 0.5-billion-parameter model) is worth the added computing cost, now that there is a real, specific signal worth trying to strengthen, rather than an open question of whether any signal exists at all.

PART VII - THE REAL-DETECTOR STRESS TEST (added 2026-10-21)

Part VI's top-priority next step was: replace the oracle ground-truth object boxes with the project's own trained detector's real predictions, and see whether the pedestrian-counting finding survives. This part reports that test, honestly, including that it mostly did not go the way Part V's result would have predicted.

What the detector actually looks like, per class

Before touching the language model, the detector's predictions were compared directly against ground truth on 500 sampled validation keyframes, separately for each class (this had not been done before - Part V only reported a single combined AP number per class, not a precision/recall breakdown):

class	real objects/keyframe (avg)	detected/keyframe (avg)	precision	recall
car	10.6	7.0	0.57	0.37
pedestrian	5.3	1.2	0.49	0.11

Pedestrians are caught more than three times less often than cars. This matters because it tells you in advance which class's results are most at risk from switching to real detections - exactly the class the headline Part V finding depended on.

First attempt: swap in the real detections, change nothing else

Using the detector's predictions (object tokens still picking the "8 nearest objects overall," same code as before) in place of ground truth, on the same 60k/6k question subsample used throughout Part V:

	oracle boxes	detected boxes
final accuracy	0.558	0.542
wrong-frame gap	+0.075	+0.043
car-counting	0.37	0.37
pedestrian-counting	**0.45**	**0.36**
speed judgment	0.73	0.72

Car-counting and speed judgment barely moved. Pedestrian-counting's margin over its 0.34 majority baseline shrank from a clear +0.11 to a marginal +0.02 - with only 321 pedestrian-counting questions in the validation set, that margin is no longer reliably distinguishable from noise (the per-condition uncertainty at this sample size is roughly +/-0.03).

Two cheap fixes, tried and isolated

Two specific, plausible explanations were tested, each changed independently so the effect of each could be seen on its own:

1. **Slot crowding**: the object-token code always keeps the 8 nearest objects regardless of class. Since cars outnumber pedestrians in the detections several-to-one, pedestrians could be getting crowded out of their slots even when correctly detected. **Fix**: reserve 4 slots for the nearest cars and 4 for the nearest pedestrians, instead of a shared pool.
2. **Detection recall**: the 0.3 confidence cutoff may filter out real pedestrians disproportionately, since they are a harder radar target with generally lower scores than cars. **Fix**: lower the cutoff to 0.15, which roughly doubled measured pedestrian recall (0.11 -> 0.21) - at the cost of precision collapsing to 0.13 (most "pedestrian" detections at this looser threshold are false positives), with no confidence-score feature on the token yet to let the model discount a shaky detection.

config	final accuracy	wrong-frame gap	car	pedestrian	speed
oracle boxes	0.558	+0.075	0.37	**0.45**	0.73
detected, 0.3 threshold, unbalanced slots	0.542	+0.043	0.37	0.36	0.72
detected, 0.3 threshold, balanced slots (fix 1)	0.519	+0.039	0.35	0.37	0.68
detected, 0.15 threshold, balanced slots (fix 1+2)	0.546	+0.049	0.36	**0.36**	0.69

Neither fix recovered the pedestrian-counting result. It stayed flat at 0.36-0.37 across all three real- detector configurations, regardless of slot allocation or detection threshold. Balancing slots alone made every other number slightly worse too, for a negligible pedestrian gain - evidence that slot crowding was not the dominant problem (reserving slots for pedestrians just reserves mostly-empty slots, since there usually aren't enough real pedestrian detections to fill them, while taking capacity away from cars that had plenty to offer). Lowering the threshold raised overall accuracy and the wrong-frame gap somewhat (likely via other question types, not pedestrians specifically) but left the pedestrian number exactly where it started - the recall gain and the precision loss appear to roughly cancel out, which is the expected result of giving a model more real detections mixed with more false ones, with no way yet to tell them apart.

What this means for the paper, stated plainly

Car-motion counting and speed judgment are **robust to real, noisy detections** - both stayed close to their oracle-boxes numbers across every configuration tested. The paper can state these as confirmed findings.

Pedestrian-motion counting - Part V's single cleanest, most specific piece of evidence for the "radar matters most where it should" thesis - is **not yet confirmed under realistic conditions**. It held up under oracle ground truth and has not yet

held up with the project's own detector, and two cheap, well-motivated fixes both failed to change that. This should be reported as an open question, not folded into the headline claim as if settled. See the updated Part VI priority list below.

A third fix, also tried: the detection confidence score (added 2026-10-21)

The top-priority item from the original version of this section - adding the detector's own confidence score as an explicit object-token feature, so the model could learn to trust a strong detection more than a weak one - has since been implemented and tested. `motion_tokens.py`'s per-slot feature vector grew from 8 to 9 numbers (`[is_car, is_pedestrian, x, y, vx, vy, speed, has_radar_points, confidence]`); `confidence` is the detector's own sigmoid score for a detected box, or 1.0 for an oracle/ground-truth box. `make_detections_cache.py` now keeps the score column instead of discarding it, and `RadarLLM`'s motion projector input widened to match.

All three box-source configurations were retrained with the new 9-dimension token:

config (with confidence feature)	final accuracy	wrong-frame gap	car	pedestrian	speed
oracle boxes	0.570	+0.081	0.40	0.51	0.72
detected, 0.3 threshold, balanced slots	0.541	+0.048	0.37	0.36	0.69
detected, 0.15 threshold, balanced slots	0.553	+0.050	0.39	0.35	0.68

Pedestrian-counting still did not move. It is flat at 0.35-0.36 for both real-detector configurations - essentially unchanged from the pre-confidence numbers (0.37 and 0.36 respectively) and nowhere close to the oracle-boxes result. Giving the model a trust signal for each detection made no measurable difference, even at the 0.15 threshold where more than three-quarters of the "pedestrian" detections are false positives. (The oracle row's jump to 0.51 is not attributable to the new feature in any meaningful sense - the confidence value for an oracle box is the constant 1.0 on every single token, so it carries no information; the move from 0.45 to 0.51 is run-to-run training variance, a reminder of exactly why Part VI item 3 - multi-seed confidence intervals - still matters before any of these specific numbers go in the paper.)

Car-counting improved somewhat across both real-detector configurations (0.37-0.39, up from 0.35-0.37), but this reads as general extra model capacity/signal from the added feature dimension rather than anything pedestrian-specific, since the one metric the fix directly targeted did not respond.

Updated priorities (supersedes Part VI's list where they overlap)

1. **Retrain the detector itself**, with more epochs and a real precision/recall sweep instead of a guessed threshold. This is now the only one of the three cheap fixes not yet tried, and the only remaining explanation consistent with the evidence: pedestrian recall (0.11 at the current operating point) is weak enough that no amount of downstream token-handling cleverness - slot allocation, detection threshold, or now confidence awareness - has been able to compensate for it. All three were tried, independently, and all three left the pedestrian number in the same narrow 0.35-0.37 band.
2. Everything else in Part VI's list (seeds/confidence intervals, scaling the training budget, the larger LLM question) still stands, now informed by this result: **the pedestrian-counting problem is very likely a detector-quality problem, not an architecture or token-design problem, and should be treated as such before any further token-side changes are attempted.**

The fourth fix, also tried: retraining the detector itself (added 2026-10-22)

The one remaining item from the list above - retraining the detector properly, with more epochs and a real precision/recall sweep instead of a guessed threshold - has now been tried too, and it closes the question rather than opening a new direction.

The original detector was trained for 8 epochs. It was retrained from scratch for 20 epochs (2.5x), with per-epoch checkpointing added to `train.py`/`train_radar.py` first (a crash/hang safety net that was missing before - a long detector run had no mid-run recovery point until now). The result:

	original (8 epochs)	**retrained (20 epochs)**
car AP@2m	0.385	0.378 (peaked 0.383 at epoch 14)
pedestrian AP@2m	0.120	0.109 (peaked 0.118 at epoch 11)

Both numbers **plateaued by epoch 7-9** and merely oscillated within noise for the remaining 11-13 epochs of training. More training bought nothing measurable, for either class.

A full precision/recall sweep (7 thresholds, 0.05 to 0.40, 500 sampled val keyframes, same methodology as the per-class diagnostic earlier in this Part) was also run on the retrained detector, specifically to check whether a better pedestrian operating point exists anywhere on the curve that the original single-threshold choice might have missed:

threshold	**pedestrian precision**	**pedestrian recall**
0.05	0.041	0.267
0.15	0.120	0.177
0.30	0.471	0.104
0.40	0.679	0.081

There is no good point on this curve. Recall never exceeds roughly 27%, and only at a precision of 4% (96% of "pedestrian" detections at that setting are false alarms). This is not a threshold-tuning problem - the model's underlying ability to *separate* real pedestrians from noise and clutter is weak across the entire operating range, not just at the one threshold used throughout this project.

All four candidate fixes for the pedestrian-counting problem have now been tried, independently, and all four failed: class-balanced object-token slots, a lower detection threshold, a confidence-score token feature, and now 2.5x more detector training with a full PR sweep. This is a strong, convergent result pointing to one conclusion: **the bottleneck is not the language model's architecture, the object-token design, or the amount of detector training - it is the information content of cheap, sparse automotive radar for a low-radar-cross-section, non-metallic, irregularly-shaped target like a pedestrian.** This reading is independently corroborated by a contemporary paper reviewed alongside this result (RFusion4D, Liang et al., IEEE OJITS 2026): even with a far higher-resolution 4D imaging radar, 2 million training frames, a dedicated signal-domain denoising module, and confidence-aware temporal fusion, their reported pedestrian AP under adverse weather is only 7.08% (up from a 5.47% camera-only baseline) - a real relative improvement, but a strikingly low absolute number from a much better-resourced system. Small, low-reflectivity, vulnerable-road-user targets appear to be a persistent, cross-project limitation of radar as a sensing modality, not an artifact of this project's specific choices.

Practical consequence: the retrained detector (`radar_pillar_trainval_v2.pt`) is not adopted as the project's checkpoint going forward - it performs statistically indistinguishably from the original (`radar_pillar_trainval.pt`) on both classes, so there is no reason to re-run the downstream language-model training on its predictions. The original checkpoint and all of Part VII's earlier results stand as the reference end-to-end numbers.

Where this leaves the paper's claims: the honest, current, and now well-tested position is: radar's contribution to car-motion counting and speed judgment is real and robust, confirmed under both oracle and realistic (real-detector) conditions. Radar's contribution to pedestrian-motion counting is real and strong under oracle conditions only; under realistic conditions it is blocked by a detection-recall ceiling that this project has now shown, through four independent and well-isolated attempts, is not fixable by token design, detection thresholding, confidence-awareness, or additional detector training - and is plausibly a near-fundamental limit of the sensor itself for this target class, not a limitation of this specific pipeline. A natural, honest framing for future work (not yet attempted) is suggested by this finding rather than contradicted by it: for vulnerable road users specifically, camera should likely lead and radar assist, the reverse of the framing that holds for vehicles - rather than one blanket "radar-led" story applied uniformly across every object class.

APPENDIX: CODEBASE MAP

File	Purpose
`pillar_dataset.py`	radar sweep loading, multi-sweep accumulation, hand-crafted pillarization, `RadarPillarDataset`
`bev_targets.py`	CenterNet target generation; `get_boxes`, `get_velocities` (ground-truth, used for motion-question generation and object tokens)
`model.py`	`PillarEncoder`, `BEVBackbone` (U-Net), `RadarPillarNet`
`train.py` / `train_radar.py`	detector training loop, mirror augmentation, AP evaluation
`make_points_cache.py` / `make_bev_cache.py` / `make_camera_cache.py` / `make_motion_cache.py`	build the four on-disk feature caches (radar points, dense BEV features, CLIP camera features, motion tokens), all version-aware (mini or trainval)
`motion_tokens.py`	explicit per-object motion token extraction; class-balanced slot allocation (added Part VII)
`make_detections_cache.py`	runs the trained detector over all keyframes and caches its real (non-oracle) predicted boxes, for the Part VII realism test
`make_motion_qa.py`	auto-generates the 3 new motion/speed question types from ground-truth velocities; version-aware
`qa_data.py`	`QADataset`, `LazyFeats` (on-disk lazy feature loading - the memory fix from Phase E)
`qa_model.py`	`RadarLLM` - the 3-modality LLM wrapper
`train_qa.py`	language-model training: subsampling, LoRA/projector optimizer groups, per-epoch checkpointing, wrong-frame control
`evaluate_all.py` / `evaluate_conditions.py`	cross-model and condition-split evaluation, reusing a trained checkpoint and reconstructing its exact validation subsample
`chat_app.py`	a Gradio app for interactively asking the trained models questions in a browser
`run_trainval_pipeline.ps1` / `run_experiments.ps1` / `run_motionqa_ablations.ps1`	unattended multi-step runners (wait for a dependency to finish, log every step, keep going even if one step fails)

Notebook	Covers
`01_radar_pillars.ipynb`	radar pillars, the detector, mirror augmentation (Phase A)
`02_radar_language.ipynb`	the first radar-only language pass, on mini (Phase B)
`03_camera_motion_fusion.ipynb`	camera integration, the motion-question negative result, object tokens, on mini (Phases C-D)
`05_trainval_motion_ablation.ipynb`	the full six-model comparison and day/night/rain results, at full trainval scale (Phases G-H)
`06_detector_realism_ablation.ipynb`	the real-detector stress test: per-class precision/recall, oracle-vs-detected boxes, the two isolation-test fixes and their negative result (Part VII)